

Seguridad del canal

Último apartado del módulo

Tu canal /ws-actividad ya emite datos reales, conectado a ActividadService desde la Actividad 4.2.

Hoy afrontas algo que has dejado pendiente a propósito: qué implica en seguridad haberlo abierto sin autenticación.

El aviso de seguridad del handshake

En la Actividad 4.2 has abierto `/ws-actividad` con `permitAll()` — sin exigir ninguna autenticación. Pero `GET /api/v1/actividad` (la misma información, por REST) exige rol ADMIN.

¿Puede un usuario anónimo suscribirse al topic y ver en vivo lo que por REST está protegido?

Sí. El handshake de WebSocket, tal como lo has configurado, no lleva el JWT por ningún sitio — un cliente que ni siquiera ha hecho login puede conectar y suscribirse sin problema.

Las opciones frente a esto

1. Restringir el handshake: exigir un token válido antes de aceptar la conexión.
2. Filtrar qué se emite: dejar el canal abierto, pero no mandar por él nada sensible.
3. Asumirlo y documentarlo como decisión consciente para un canal de demostración.

Aquí no vale quedarse solo en detectar y documentar

Terminar el módulo con un agujero de seguridad real, detectado y descrito por escrito pero sin corregir, transmite el mensaje contrario a todo lo construido. La remediación mínima es obligatoria, no opcional.

La remediación mínima: JWT en el handshake

La forma más simple de exigir el token en el handshake: pasarlo como parámetro de consulta en la propia URL de conexión, y validarlo con un interceptor antes de aceptar.

```
const client = new StompJs.Client({  
  brokerURL: `ws://localhost:8080/ws-actividad?token=${token}`  
});
```

Ese parámetro no lo añade el navegador por su cuenta: lo pone el propio cliente, al construir la URL de conexión — no hay ninguna cookie ni sesión de por medio.

Qué es un interceptor

Un gancho que Spring ejecuta justo antes (y justo después) de que la conexión WebSocket se establezca, y en el que tu código decide, con un simple true/false, si esa conexión sigue adelante o se corta ahí mismo.

HandshakeInterceptor tiene dos métodos: `beforeHandshake` (donde decides si la conexión sigue) y `afterHandshake` (se ejecuta después de establecerse, útil para logging o limpieza — hoy no hace falta).

JwtHandshakeInterceptor (1/2): beforeHandshake

```
@Component
@RequiredArgsConstructor
public class JwtHandshakeInterceptor implements HandshakeInterceptor {

    private final JwtDecoder jwtDecoder;

    @Override
    public boolean beforeHandshake(ServerHttpRequest request,
        ServerHttpResponse response, WebSocketHandler wsHandler,
        Map<String, Object> attributes) {
        String token = extraerToken(request.getURI().getQuery());
        if (token == null) {
            response.setStatus(HttpStatus.UNAUTHORIZED);
            return false; // rechaza el handshake
        }
        try {
            jwtDecoder.decode(token); // lanza excepción si no es válido
            return true;
        } catch (JwtException e) {
            response.setStatus(HttpStatus.UNAUTHORIZED);
            return false;
        }
    }
}
```

JwtHandshakeInterceptor (2/2): afterHandshake y el token

```
@Override
public void afterHandshake(ServerHttpRequest request,
    ServerHttpResponse response, WebSocketHandler wsHandler,
    Exception exception) {
    // no hace falta nada aquí para este caso
}

private String extraerToken(String query) {
    if (query == null) return null;
    for (String param : query.split("&")) {
        if (param.startsWith("token=")) return param.substring(6);
    }
    return null;
}
}
```

`beforeHandshake` recibe la petición en crudo: no hay ningún `@RequestParam` que lo resuelva solo, así que `extraerToken` recorre la query string a mano, param a param, hasta encontrar el que empieza por "token=".

Conectando el interceptor a la configuración

```
@Configuration
@EnableWebSocketMessageBroker
@RequiredArgsConstructor
public class WebSocketConfig implements WebSocketMessageBrokerConfigurer {

    private final JwtHandshakeInterceptor jwtHandshakeInterceptor;

    @Override
    public void registerStompEndpoints(StompEndpointRegistry registry) {
        registry.addEndpoint("/ws-actividad")
            .setAllowedOriginPatterns("*")
            .addInterceptors(jwtHandshakeInterceptor);
    }
    // configureMessageBroker sigue exactamente igual
}
```

Al llevar `@RequiredArgsConstructor` en ambas clases, Spring construye e inyecta las dos piezas solo — igual que reutiliza el mismo `JwtDecoder` ya configurado desde el Tema 2 para validar tokens en el resto de la API.

Qué comprueba exactamente, y qué límite tiene

`jwtDecoder.decode(token)` comprueba que el token es válido, no que su dueño tiene rol ADMIN

Cierra la puerta a quien no ha hecho login en absoluto — el agujero real detectado en el Paso 1 de la Actividad 4.3 — pero no reproduce del todo la misma política que `GET /api/v1/actividad`: cualquier usuario autenticado, no solo uno con rol ADMIN, sigue viendo el canal en vivo.

Pasar el token en la URL (`?token=...`) puede quedar registrado en logs del servidor, proxies o monitorización. Para esta práctica local es una solución sencilla y observable; en producción convendría un mecanismo que no exponga el token en la propia URL.

Depuración y documentación

WebSocket no aparece en Swagger, así que la nueva ruta no queda documentada automáticamente. Pero /ws-actividad es, ante todo, una ruta más con su propia política de acceso: añádela a docs/seguridad.md (Actividad 2.5), igual que las demás.

Ruta	Verbo	Quién puede
/ws-actividad (handshake, Upgrade: websocket)	GET	Cualquiera con un JWT válido — no distingue rol

Es el mismo documento en el que ya clasificaste el resto de rutas del proyecto — no hace falta ningún criterio distinto para esta, aunque no sea HTTP tradicional.

Ideas clave

- Un canal abierto sin autenticación puede exponer, por WebSocket, información que por REST está protegida — una incoherencia real que hay que resolver, no solo señalar.
- Un HandshakeInterceptor puede rechazar la conexión WebSocket antes de establecerse, validando un token pasado como parámetro de consulta.
- Esa validación comprueba que el token es válido, no el rol de su dueño: cierra el agujero de autenticación, pero no reproduce la política de solo-ADMIN del REST — una remediación mínima, con ese límite consciente.
- Documentar y detectar un agujero de seguridad no sustituye corregirlo — la remediación mínima es parte obligatoria de la entrega.
- WebSocket no aparece en Swagger — documentas la ruta a mano, en el mismo docs/seguridad.md del Tema 2, no en un fichero aparte.